

Prompt 6 — Tablas oro y emisión Arrow (F3)

Repo de especificación: github.com/leanchoi/leanchoi, rama `claude/esquel-data-ecosystem-integration-qbxuv2`.
Leé `AGENTS.md` y `docs/03-pipeline-datos.md` §4-§8.

OBJETIVO

Que el ETL emita las tablas nuevas junto a las 24 existentes, sin alterarlas.

EL CRITERIO MÁS IMPORTANTE DE ESTA TAREA

Las 24 tablas preexistentes deben salir BYTE-IDÉNTICAS. Verificalo con `sha256sum` sobre una corrida previa y otra posterior al cambio. Si una sola difiere, algo está mal y hay que encontrarlo antes de seguir. No lo dejes para el final.

SUPERFICIE DE CAMBIO

<code>etl/build.py</code>	+ bloques <code>try/except</code> por dataset nuevo <- ÚNICO archivo existente
<code>etl/emit.py</code>	+ validación de esquema y orden físico de las tablas nuevas
<code>etl/gold_aereos.py</code>	nuevo
<code>etl/gold_ota.py</code>	nuevo
<code>oit_api/main.py</code>	NINGÚN cambio – <code>/api/datos/{tabla}.arrow</code> ya es genérico

TABLAS (grano, filas estimadas, carga) en `docs/03` §4. Respetá los prefijos `air_ota_x_ext_`.

REGLA DE DISEÑO que evita el error más caro:

NUNCA cruces la dimensión de tipología con la de anticipación.

10 destinos x 730 noches x 10 buckets x 5 tipologías = 365.000 filas y ~4 MB, para responder preguntas que nadie hace. Dos tablas separadas suman 65.000 filas y <1 MB. Cuando una tabla oro crezca, la solución es QUITAR UNA DIMENSIÓN, no comprimir más.

PRESUPUESTO DE BYTES (`docs/03` §5) – es un control, no una guía:

por tabla 2 MB · carga inicial eager 3 MB · por sección lazy 2,5 MB

Compresión: NO uses la interna de Arrow IPC (soporte desperejo entre implementaciones, es una trampa de compatibilidad conocida). Usá:

- Arrow IPC sin comprimir + Content-Encoding `br/gzip` desde FastAPI, para las eager.
- Parquet con ZSTD interno para las lazy grandes; DuckDB-WASM lo lee nativo.

OPTIMIZACIÓN DE MAYOR IMPACTO, y cuesta un ORDER BY:

emitir físicamente ordenado por (destino, fecha). DuckDB guarda min/max por row group, así que un filtro de rango saltea grupos enteros sin leerlos.

Además: fechas como DATE (jamás VARCHAR), códigos como diccionario Arrow -> ENUM, métricas en float32 salvo que se necesite precisión, bucket de anticipación precalculado.

AISLAMIENTO DE FALLOS (I3)

`try/except` por tabla. Una que falla se marca `disponible:false` en `meta.json` y EL BUILD CONTINÚA. Probalo: simulá la falla de CADA fuente nueva, una por una, y verificá que el

ETL termina en verde y emite las 24 tablas.

CRITERIOS DE ACEPTACIÓN

1. Las 24 tablas existentes byte-idénticas (sha256sum). Innegociable.
2. Cada tabla nueva dentro de su presupuesto de bytes; reportá los tamaños reales.
3. Emisión ordenada físicamente por (destino, fecha).
4. meta.json con disponibilidad, cobertura y schema_version por dataset.
5. Falla simulada de cada fuente nueva -> ETL en verde.
6. Los .arrow nuevos cargan en DuckDB-WASM con los tipos esperados (verificalo de verdad, no lo asumas).
7. validar_columnas() del catálogo activo sobre todas las tablas nuevas (I11).